

TP1 : Filtrage de données pour nos amis les spammeurs

1 Une première fonction d'analyses des numéros de téléphone (45mn)

Créez un nouveau fichier nommé *TP1-Spam.py*.

Vous allez, dans la suite, devoir manipuler un fichier de données de numéros de téléphone français, mais récupérés au fil du temps par différents stagiaires qui les ont souvent écrits n'importe comment... Les numéros pourront avoir des formats très variés comme 01 23 45 67 89, +331 23 45 67 89, 0033 123 456 789, 01 23 (456) 789, 01-23-45-67-89, etc. Un numéro de téléphone sera considéré comme correct s'il commence par un 0 et possède dix chiffres.

Il vous faudra, au fil des questions, proposer une fonction **format_phone_number** qui prendra en entrée une chaîne de caractères représentant un numéro de téléphone, et renvoyant en sortie une paire d'éléments : le premier élément sera un booléen permettant de dire si le numéro de téléphone a un format correct, et le second sera une chaîne de caractères représentant le numéro de téléphone formaté sous la forme 01-23-45-67-89 (des paires de chiffres séparées par des tirets). L'objectif de votre fonction sera donc de formater, si cela est possible, le numéro de téléphone.

Pour coder cette fonction, voici les différentes étapes à suivre :

1. Créez la fonction **format_phone_number** qui commencera par retirer tout ce qui n'est pas un chiffre ou le symbole '+' de la chaîne transmise, à l'aide d'une compréhension de liste.
*Indice : Construisez d'abord une liste, à partir de la chaîne transmise, consistant simplement en les éléments numériques ou le symbole '+' de la chaîne. Transformez ensuite cette liste en chaîne de caractères à l'aide de la méthode **join** de la classe **string**.*
2. Si la chaîne commence par +33 ou 0033, retirez ce bloc de chiffre et remplacez le par un simple 0. Ensuite, à l'aide d'une compréhension de liste, pensez à retirer tous les symboles '+' de la chaîne (ils pourraient apparaître au centre de la chaîne).
3. Si la chaîne ne commence pas par un 0 ou ne fait pas 10 chiffres de long, on retourne un booléen False accompagné de la chaîne nulle (None).
4. A l'aide d'une compréhension de liste, transformez la chaîne selon le format demandé et retournez la avec un booléen Vrai.

*Indice : Commencez par construire une liste constituée des éléments de la chaîne de caractères pris deux par deux. Puis, à l'aide de la fonction **join**, rassemblez les éléments de cette liste avec des tirets.*

Testez votre fonction en écrivant un code principal, dans un main bien écrit, qui demande, à l'aide de la fonction **input** un numéro de téléphone à l'utilisateur puis déclenche votre fonction. Ces numéros doivent être acceptés comme étant valides : 01 234 (567) 89, +(33) 6 58 47 25 14, (0033)2 456 (789) 25. Ces numéros ne doivent pas être valides : +033 1 56 74 89 65, 33 5 89 74 12 22, 06 45 12 78.

2 Analyser un fichier complet de numéros (30mn)

Vous trouverez, avec les fichiers du TP, un fichier texte nommé *liste_numeros.txt*. Ce fichier regroupe de nombreux noms et des numéros de téléphone récupérés par différents stagiaires au fil du temps.

Comme vous pouvez vous en apercevoir, les numéros de téléphone n'ont pas tous le même format, aussi aurez-vous besoin de votre fonction codée précédemment afin de filtrer les numéros de téléphone et leur donner un format correct.

Votre travail consistera à construire un nouveau fichier, présenté de la même façon, mais dans laquelle les numéros de téléphone seront tous formatés correctement. Si un numéro de téléphone n'est pas correct, ce numéro et le nom associé ne devront pas apparaître dans votre fichier de sortie.

Vous êtes libres de procéder comme vous le souhaitez, mais voici quelques étapes à suivre si vous êtes perdu :

1. Vous utiliserez la fonction **open** de Python pour ouvrir le fichier en lecture. Vous penserez à gérer les exceptions de fichier introuvable (**FileNotFoundError**, avec un bloc **try/except**), et pourrez utiliser l'instruction suivante pour mettre fin à l'exécution de votre programme :

Code Python

```
exit(0)
```

2. Vous utiliserez la méthode **reader** de la classe **csv** pour lire le fichier d'entrée, et récupérer un objet **_csv_reader** que vous convertirez en liste simplement avec la fonction **list**.
3. Vous obtiendrez ensuite une liste de paire d'éléments <Nom, Numero>, que vous casserez en deux sous listes à l'aide de **zip**.
4. A partir de là, vous devrez construire une nouvelle liste d'éléments <Nom, Numero> où n'apparaissent que les bons numéros de téléphone, en une seule ligne de code, à l'aide d'une compréhension de liste. *Indice : Pour vous aider, pensez à ce que pourrait donner la liste résultant d'un **map** de la fonction codée en première partie sur la liste de numéros de téléphone. Pourriez-vous zipper cette liste avec les deux autre obtenues précédemment ?*
5. Toujours grâce à la fonction **open**, ouvrez un fichier *output.txt* en écriture et créez un objet **csv_writer** grâce à la méthode **_csv_writer** de la classe **csv**. Ensuite, vous appellerez la méthode **writerows** de cet objet, pour écrire votre liste obtenue précédemment dans le fichier de sortie.

Avez-vous bien un fichier de sortie avec des noms et des numéros de téléphone ? Vous ne devriez avoir que 63 noms dans votre liste (4 de moins que dans la liste originale). Si tout va bien, Nola Baker ne devrait plus faire partie de votre fichier de sortie. Avez-vous bien la même présentation de fichier que le fichier d'entrée ? Y a-t-il bien un espace entre les deux points séparateurs (:) et les numéro de téléphone ?

3 Facultatif : poursuivre les menteurs et les familles

1. On souhaiterait obtenir, à l'écran, les noms des personnes ayant transmis un mauvais numéro de téléphone. Pouvez-vous modifier votre programme afin d'afficher ces scélérats ?
2. On souhaite maintenant trouver, dans la liste des bons numéros de téléphone, les personnes qui feraient partie d'un même foyer et qui auraient donc le même numéro de téléphone. On souhaite obtenir une liste de listes : chaque liste de la liste principale contiendrait les noms des personnes ayant un même numéro de téléphone. Vous devriez obtenir, si tout est bon, deux noms pour une première même famille, et trois noms pour une seconde même famille.

*Indice : Commencez par extraire la liste *l* de tous les bons numéros de téléphone. Puis, construisez la liste *k* de tous les numéros de téléphone apparaissant plusieurs fois dans *l* (chaque numéro doit n'apparaître qu'une seule fois). Puis, en une seule ligne de code, réfléchissez à comment extraire tous les noms des personnes ayant comme numéro de téléphone le premier élément de *k*. Enfin, toujours en une seule ligne de code, essayez d'extraire la liste de listes de personnes ayant le même numéro de téléphone.*